

Aula 09 - Simulação de Sensores Digitais e Analógicos

Descrição: Simulação de leitura de sensores (temperatura, luminosidade, presença) e condicionamento de sinal.

Introdução

A aula de hoje é o ponto de partida para a criação de sistemas IoT, marcando o início da nossa estação de telemetria. O objetivo é compreender como um microcontrolador interage com o mundo real. Para que um sistema tome decisões, ele primeiro precisa "sentir" o ambiente. Nesta aula, exploraremos como capturar estímulos físicos (luz, calor, movimento e distância) e transformá-los em dados computacionais dentro da linguagem C. A lógica da aula seguirá uma progressão clara: entender o conceito físico, compreender o componente no simulador e, em seguida, aplicar o código em blocos isolados antes do desafio final.

Etapa 1: O Universo dos Sensores

Para que um sistema de software interaja com o mundo físico, utilizamos **sensores**, que são transdutores capazes de converter grandezas físicas em sinais elétricos. Na arquitetura do Arduino Uno (ATmega328P), dividimos esses sensores em duas categorias principais de leitura:

1. **Sensores Digitais:** Trabalham com lógica booleana discreta. O sinal elétrico é interpretado apenas como **HIGH** (Nível Alto, 5V, ou 1) e **LOW** (Nível Baixo, 0V, ou 0).
 - **Exemplo:** Um sensor de presença PIR. Ele responde de forma binária: há movimento (**HIGH**) ou não há movimento (**LOW**).
 2. **Sensores Analógicos:** O mundo real não é feito apenas de 0 e 1; ele é contínuo. Sensores analógicos fornecem uma variação contínua de tensão entre 0V e 5V. Para que o C entenda isso, o Arduino possui um Conversor Analógico-Digital (ADC) de 10 bits, que "fatia" essa tensão em 1024 degraus. Portanto, uma leitura analógica no pino **A0** entregará um número inteiro variando de 0 a 1023.
 - **Exemplo:** O sensor de temperatura TMP36 e o resistor dependente de luz (LDR).
 3. **Sensores Baseados em Tempo de Pulso:** Operam com sinais digitais, mas a informação da grandeza física não está no nível lógico imediato (0 ou 1), e sim no tempo de duração em que um pulso permanece em nível lógico alto.
 - **Exemplo:** O sensor ultrassônico HC-SR04. Ele emite um pulso de som e aciona um pino digital durante o tempo exato em que a onda sonora viaja até um obstáculo e retorna (eco), permitindo calcular a distância.
-

Prática 1: Leitura Bruta dos Sensores

Passo 1: Preparação do Ambiente

1. Acesse a plataforma [Tinkercad](#).
2. Crie um novo projeto no módulo de **Circuitos** chamado "Aula 09 - Sensor".

Passo 2: Seleção dos Componentes

Adicione os seguintes componentes à área de trabalho:

- **1x** Arduino Uno
- **1x** Protoboard (*Placa de ensaio*)
- **1x** Sensor de Presença PIR (Digital)
- **1x** Sensor de Temperatura TMP36 (Analógico)
- **1x** Resistor Dependente de Luz - LDR (Analógico)

Passo 3: Montagem do Circuito Eletrônico (Hardware)

1. Conexão do Sensor PIR (Digital):

- Conecte o pino de sinal do PIR ao **Pino Digital 2** do Arduino.

2. Conexão do Sensor TMP36 (Analógico):

- Conecte o pino central de saída (Vout) do TMP36 ao **Pino Analógico A1** do Arduino.

3. Conexão do Sensor LDR (Divisor de Tensão):

- Conecte o outro terminal do LDR ao **Pino Analógico A0** do Arduino.

Atenção: Pesquise nas suas fontes de aula as ligações corretas dos pinos. Esta pesquisa faz parte da avaliação da atividade.

Passo 4: Programação do Arduino (Código-Fonte)

1. Clique no botão **"Código"** no canto superior direito do Tinkercad e altere o modo para **Texto**.
2. Insira o código do arquivo aula09sensor.c dos anexos da aula na aula:

Passo 5: Simulação e Visualização dos Dados

1. Com o código inserido, clique no botão **"Monitor Serial"** na parte inferior do painel de código para abrir a janela de visualização de texto.
2. Clique no botão **"Iniciar Simulação"** no topo da página.
3. Clique sobre os sensores no simulador para alterar a iluminação (LDR), a temperatura (TMP36) ou simular movimento (PIR). Observe os números alterando em tempo real na tela do Monitor Serial.

Dica de Análise:

- **Leitura Digital (PIR):** Apresentará apenas 0 (sem movimento / LOW) ou 1 (movimento detectado / HIGH).
- **Leitura Analógica (LDR e TMP36):** Apresentará valores numéricos entre 0 e 1023. Note que o valor exibido do TMP36 (ex: 153) não é a temperatura real em graus Celsius ainda, mas sim o valor elétrico "fatiado" pelo Conversor Analógico-Digital (ADC) de 10 bits do Arduino.

Explicação do código:

O Bloco de Código 1 tem a função de ler as informações "cruas" dos sensores e exibi-las na tela do computador, dividindo-se em três partes principais:

1. Definição dos Pinos (Identificação das Ligações)

No início do código, é indicado ao Arduino onde cada sensor se encontra fisicamente ligado:

- `const int PINO_LDR = A0;` O sensor de luz (LDR) está ligado à entrada analógica **A0**.
- `const int PINO_TEMP = A1;` O sensor de temperatura (TMP36) está ligado à entrada analógica **A1**.
- `const int PINO_PIR = 2;` O sensor de presença (PIR) está ligado ao pino digital **2**.

2. Configuração Inicial (setup)

Esta seção corre **uma única vez** assim que o Arduino é ligado ou reiniciado:

- `Serial.begin(9600);`: Abre o canal de comunicação entre o Arduino e o computador a uma velocidade de 9600 bits por segundo.
- `pinMode(PINO_PIR, INPUT);`: Configura o pino digital do sensor PIR como **entrada** para que o Arduino possa "escutar" o sinal do sensor. *(Os pinos analógicos A0 e A1 já funcionam como entrada por defeito).*

3. Ciclo Principal (loop)

Esta seção é executada continuamente em ciclo:

1. Leitura dos Sensores:

- `analogRead();`: Lê o valor dos sensores LDR e TMP36. Por serem analógicos, convertem o sinal num valor numérico de **0 a 1023**.
- `digitalRead();`: Lê o sensor PIR. Por ser digital, devolve apenas dois estados: **1** (movimento/presença detetada) ou **0** (sem movimento).

2. Exibição dos Dados:

- Os comandos `Serial.print()` e `Serial.println()` imprimem os valores formatados no **Monitor Serial** da IDE do Arduino numa única linha: `LDR (Luz Bruta): [valor] | TMP36 (Temp Bruta): [valor] | PIR (Presenca): [0 ou 1]`

3. Pausa:

- `delay(1000);`: Faz o microcontrolador aguardar **1 segundo (1000 ms)** antes de reiniciar o ciclo de leitura.

ETAPA 2: Condicionadores de Sinal

Explicação: O dado "bruto" (0 a 1023) gerado pelo ADC não tem utilidade direta para o usuário ou para lógicas avançadas de software. Precisamos de **condicionamento de sinal**, que pode ser tanto uma adequação no hardware (elétrica) quanto no software (matemática).

1. Condicionamento Elétrico (Hardware):

- Um LDR altera sua resistência com a luz, mas o Arduino lê *tensão* (Volts). Para converter a variação de resistência em variação de tensão, utilizamos um circuito chamado **Divisor de Tensão**, associando o LDR a um resistor (ex: 10 k Ω) conectado ao `GND` (resistor *pull-down*).

2. Condicionamento Matemático (Software):

- **Temperatura:** O sensor TMP36 gera 10mV para cada grau Celsius. Como o ADC divide 5V em 1024 partes, nossa constante de conversão de unidade ADC para Celsius é calculada pela base `(5.0 / 1023.0) * 100.0`, o que nos dá o fator `0.488758`.
 - **Distância Ultrassônica:** O sensor HC-SR04 emite um pulso sonoro e mede o tempo de retorno em microssegundos com a instrução `pulseIn()`. Para converter esse tempo em centímetros, dividimos a duração por 58.
-

Prática 2: Implementando Funções de Conversão

Vocês vão apagar o bloco de código anterior e focar apenas na construção lógica das funções em C que transformam dados elétricos em grandezas físicas padronizadas.

Montagem base: Adicionar o sensor Ultrassônico HC-SR04 ao circuito (Trigger no pino 8, Echo no pino 7).

Bloco de Código 2 - Condicionamento Lógico de sinal:

Arquivo [aula09atuador.c](#)

Explicação do Código: Este bloco de código atua como um "motor de testes" isolado e um "tradutor" de sinais. Ele prova que a matemática das suas funções está correta antes de você conectar todos os componentes físicos. A lógica se divide nas seguintes etapas:

- **Constantes e O Tradutor (A Lógica):** A função `condicionarTemperatura` converte a leitura analógica em tensão (0 a 5V) e subtrai o *offset* de 0,5V característico do sensor **TMP36** antes de multiplicar por 100. A função `condicionarLuminosidade` usa a regra de três para transformar a leitura elétrica do LDR em porcentagem. A função `lerDistanciaUltrassonica` controla os pulsos do pino TRIG e usa a velocidade do som (dividindo o tempo por 58) para calcular centímetros exatos no pino ECHO.
- **Preparação do Sistema (setup):** O comando `Serial.begin(9600)` liga o Monitor Serial. Os comandos `pinMode` preparam exclusivamente os pinos do sensor ultrassônico (`OUTPUT` para disparar o som e `INPUT` para escutar).
- **Dados Fictícios de Teste (loop):** Como não montamos LDR e TMP36 neste teste, o código injeta valores elétricos "inventados" (512 para luz e 153 para temperatura) diretamente nas funções matemáticas, validando o condicionamento independentemente do hardware.
- **Leitura Ultrassônica Real:** Diferente da luz e temperatura, a distância é testada na prática. A função interage com o sensor HC-SR04 real na tela do simulador.
- **Exibição e Ritmo:** Os comandos `Serial.print` organizam e escrevem no console os resultados finais (Luz/Temperatura simuladas e Distância real). O `delay(1500)` pausa o sistema para facilitar a leitura.

Resumo das Modificações Realizadas

1. **Remoção de `BASE_CELSIUS`:** Substituída pelo cálculo explícito de tensão e subtração do *offset* de 0,5V do TMP36.
2. **Atualização do valor simulado de temperatura:** Alterado de 50 para 153 no `loop()`, garantindo que o teste simule uma temperatura ambiente coerente (~24.8 °C) com a escala do TMP36.
3. **Texto explicativo:** Atualizados os trechos que faziam referência ao TMP36 e ao fator de multiplicação único.

Etapa 3: O Desafio do Protagonismo

Nesta etapa prática você assume o papel de protagonista do sistema, consolidando os blocos lógicos isolados das etapas anteriores em um único firmware funcional. A transição ocorre da aprendizagem guiada para a resolução autônoma de problemas.

1. Dinâmica da Atividade

- **Unificação de Código:** Você inicia um *sketch* em branco no ambiente do Tinkercad. Ele deve declarar

globalmente os pinos (`PINO_LDR`, `PINO_TEMP`, `PINO_PIR`, `PINO_TRIG`, `PINO_ECHO`) e transcrever as funções de conversão matemática (`condicionarTemperatura`, `condicionarLuminosidade`, `lerDistanciaUltrassonica`) trabalhadas na Etapa 2.

- **Construção do Fluxo Principal:** Utilizando a estrutura básica da linguagem C em microcontroladores, você codifica a função `setup()` para inicializar a comunicação serial e definir os modos dos pinos (`INPUT` ou `OUTPUT`). Em seguida, desenvolve a rotina contínua dentro da função `loop()`.
- **Aplicação de Lógica Condicional:** Utilize as estruturas de controle de decisão (`if/else`) para analisar os dados condicionados e determinar o estado de segurança do ambiente com base no sensor PIR.

2. Especificações Técnicas Requeridas

- Os valores brutos obtidos por `analogRead()` e `digitalRead()` não podem ser impressos diretamente na tela. Eles devem ser armazenados em variáveis locais e, obrigatoriamente, passados como parâmetros para as funções de condicionamento.
- A avaliação do sensor PIR deve verificar o estado lógico. Se o pino estiver recebendo 5V, a leitura será interpretada como `HIGH` (movimento), caso contrário, será `LOW` (repouso).
- O sistema deve conter um intervalo de tempo (`delay()`) no final do ciclo principal, estabilizando a taxa de atualização dos dados no Monitor Serial.

3. O Entregável

Você deve enviar no AVA todos entregáveis em **um único documento de texto** (pdf ou docx). A entrega é composta por três frentes de avaliação:

- **Circuito Físico Virtual: (Imagem)** Montagem dos quatro sensores na *protoboard*, garantindo o barramento de alimentação (5V e `GND`) correto para o HC-SR04, PIR e TMP36, além do divisor de tensão resistivo devidamente conectado ao LDR.
- **Código-Fonte em C Polido: (Código)** O código deve apresentar indentação correta, uso de funções modulares para os cálculos e comentários pontuais (utilizando `//`) para explicar as lógicas de conversão e estruturação de dados.

Copie o código e cole somente o texto no documento de resposta da atividade.

- **Console de Saída Formatado (Monitor Serial): (Print)** O sistema deve processar as grandezas e imprimir uma linha limpa de *telemetria* a cada ciclo, formatando os dados condicionais exatamente como no exemplo abaixo:

`Temp: 25.40 C | Luz: 85% | Dist: 120 cm | Status: Área Segura`

(Saída gerada quando o valor do PIR for `LOW`)

`Temp: 25.40 C | Luz: 85% | Dist: 120 cm | Status: ALERTA: Presença Detectada`

(Saída gerada quando o valor do PIR for `HIGH`)

Tire um print da tela do Monitor Serial e anexe no documento de texto.

Encerramento e Gancho para a próxima aula

Parabéns pela conclusão do núcleo de aquisição de dados! Hoje o nosso sistema ganhou olhos, pele e ouvidos, convertendo grandezas físicas complexas em variáveis numéricas que nosso software em C domina

perfeitamente. Vocês perceberam como condicionar os sinais é essencial para obtermos dados confiáveis.

Porém, um sistema que apenas "sente" o ambiente é passivo. Na **Aula 10 - Simulação: Atuadores e Modulação PWM**, daremos vida ao projeto. O código unificado que vocês construíram agora será a base de decisão. Utilizaremos o percentual de luz que acabamos de calcular para movimentar as pás de um servomotor de forma contínua, modulando sua posição. A detecção de presença acionará a comutação de relés de potência para ligar lâmpadas reais, e o limiar de temperatura fará o chaveamento de transistores para controle de velocidade de motores CC via PWM. O código de leitura da Aula 09 vai disparar a ação mecânica da Aula 10! **Guardem bem este código, ele é o cérebro da nossa próxima montagem.**